

Titre professionnel RNCP 37827

Développeur en intelligence artificielle

Niveau 6, équivalent à Bac +3/4

[logo France compétences]

livrable E5

Surveillance et résolution d'incidents

Déboguer une application de classification d'images satellite

Passage par la voie de la formation — parcours de 19 mois achevé en mars 2026

Alexandre Laugier

mars 2026

INSÉRER ICI LA DATE DE REMISE

Table des matières

1	Contexte et description de l'application	3
1.1	Objectif	3
1.2	Stack technique	3
2	Identification et correction des bugs	3
2.1	Bug 1 — Redimensionnement absent	3
2.2	Bug 2 — Double normalisation	4
2.3	Flux de traitement corrigé	5
3	Validation par tests automatisés	6
3.1	Stratégie de test	6
3.2	Résultats	6
4	Observabilité : journalisation, alerting et monitoring	6
4.1	Journalisation	6
4.2	Alerting	7
4.3	Monitoring	7
5	Feedback loop et perspectives de ré-entraînement	7
5.1	Collecte des feedbacks	7
5.2	Stratégie de ré-entraînement sans fuite de données	7
	Références	8
	Annexes	9

1 Contexte et description de l'application

1.1 Objectif

L'application est une interface web de **classification d'images satellite** développée avec Flask. Elle permet à un utilisateur de soumettre une image et d'obtenir une prédiction parmi quatre classes : `desert`, `forest`, `meadow` et `mountain`. L'inférence repose sur un modèle CNN pré-entraîné au format Keras. Un mécanisme de feedback utilisateur alimente une base MongoDB Atlas pour permettre un futur ré-entraînement du modèle.

1.2 Stack technique

Composant	Technologie	Rôle
Backend	Flask 2.3.3 [6]	Routing, templates Jinja2, API REST
ML/IA	Keras 2.13.1 [2] + TensorFlow	Inférence CNN pré-entraîné
Persistance	MongoDB Atlas [5]	Feedbacks utilisateurs (NoSQL)
Monitoring	SQLite + Flask-MonitoringDashboard [1]	Métriques de performance
Tests	unittest (stdlib) [8]	12 tests automatisés, pipeline CI
Déploiement	Gunicorn [4] + Render (PaaS)	Serveur de production

TABLE 1 – Stack technique de l'application de classification d'images satellite.

2 Identification et correction des bugs

Deux bugs indépendants ont été identifiés dans la fonction `preprocess_from_pil()` de `app/app.py`. Le premier provoque une erreur bloquante visible dans les logs ; le second est silencieux mais tout aussi critique : il dégrade les prédictions sans lever d'exception.

2.1 Bug 1 — Redimensionnement absent

Symptôme

Au lancement d'une prédiction sur une image uploadée, le serveur lève l'exception suivante :

```
ValueError: Input 0 of layer "functional_1" is incompatible with the layer:
expected shape=(None, 224, 224, 3), found shape=(1, 600, 600, 3)
```

Le modèle Keras attend un tenseur d'entrée de forme `(None, 224, 224, 3)`, mais reçoit un tenseur `(1, 600, 600, 3)` correspondant à l'image originale non redimensionnée.

Cause racine

La fonction `preprocess_from_pil()` convertit l'image en RGB et la normalise, mais **omet l'étape de redimensionnement**. L'image est donc transmise au modèle à sa taille d'origine. Le code fautif est reproduit en Annexe A.

Correctif appliqué

La correction s'effectue en deux étapes :

1. **Extraction dynamique des dimensions** attendues par le modèle lors du chargement, pour éviter toute valeur codée en dur :

```
model = keras.saving.load_model(MODEL_PATH, compile=False)
MODEL_HEIGHT, MODEL_WIDTH = input_shape[1], input_shape[2]
```

2. **Ajout du redimensionnement** via `Image.Resampling.LANCZOS` [3] (filtre haute qualité, conforme aux recommandations Pillow ≥ 9.1).

2.2 Bug 2 — Double normalisation

Symptôme

Après correction du redimensionnement, l'application ne lève plus d'erreur mais les **prédictions sont identiques pour toutes les images** : le modèle retourne systématiquement la même classe quelle que soit l'entrée.

Cause racine

L'inspection de l'architecture du modèle [2] via `model.summary()` révèle la présence d'une couche `Rescaling` intégrée :

rescaling (Rescaling)		scale: 1/255		offset: 0.0
-----------------------	--	--------------	--	-------------

La fonction `preprocess_from_pil()` effectuait déjà une normalisation / 255.0 avant d'envoyer le tenseur au modèle. Le modèle appliquait ensuite une seconde division par 255. Les pixels étaient donc divisés par $255^2 = 65\,025$, produisant des valeurs de l'ordre de 10^{-5} .

Cette double normalisation entraîne une cascade d'effets :

1. **Activations plates** dans les couches convolutionnelles : les filtres reçoivent un signal quasi nul et ne peuvent extraire aucun motif.
2. **Perte totale d'information visuelle** : toutes les images produisent le même vecteur d'activation, quelle que soit leur apparence.
3. **Vecteur constant** après le `GlobalAveragePooling2D` : la couche dense finale reçoit un vecteur uniforme.
4. **Prédictions dégénérées** : le modèle retourne la même classe pour toutes les images (effondrement sur la classe majoritaire).

Ce bug est **silencieux** : aucune exception n'est levée, les logs n'indiquent aucune anomalie, et le modèle semble fonctionner normalement. Il n'est détectable que par l'analyse qualitative des prédictions ou par l'inspection de l'architecture du modèle.

Méthode de détection

La couche Rescaling a été identifiée en inspectant la configuration de la troisième couche du modèle en session Python interactive :

```
model = keras.saving.load_model("models/final_cnn.keras", compile=False)
print(model.layers[2].get_config())
# {'name': 'rescaling', 'scale': 0.00392156862745098, 'offset': 0.0}
# scale = 1/255 => normalisation déjà integree dans le modele
```

La valeur `scale: 0.00392...` correspond exactement à $1/255$, confirmant que la normalisation est déjà prise en charge par le modèle lors de l'inférence.

Correctif appliqué

La division `/ 255.0` a été supprimée de `preprocess_from_pil()`. Le tenseur est transmis au modèle avec des valeurs entières dans $[0, 255]$, conformément à ce qui était attendu lors de l'entraînement.

Étape	Avant correction	Après correction
Prétraitement	<code>array / 255.0</code> $\rightarrow [0, 1]$	<code>array brut</code> $\rightarrow [0, 255]$
Couche Rescaling	$[0, 1] \div 255 \rightarrow [0, 3.9 \times 10^{-3}]$	$[0, 255] \div 255 \rightarrow [0, 1]$
Signal reçu par Conv2D	$\approx 10^{-5}$ (quasi nul)	$\in [0, 1]$ (cohérent avec l'entraînement)
Prédictions	Identiques pour toutes les images	Différenciées selon le contenu

TABLE 2 – Impact de la double normalisation avant et après correctif.

La version finale corrigée de la fonction, intégrant les deux correctifs (redimensionnement + suppression de la normalisation manuelle), est disponible en Annexe A.

2.3 Flux de traitement corrigé

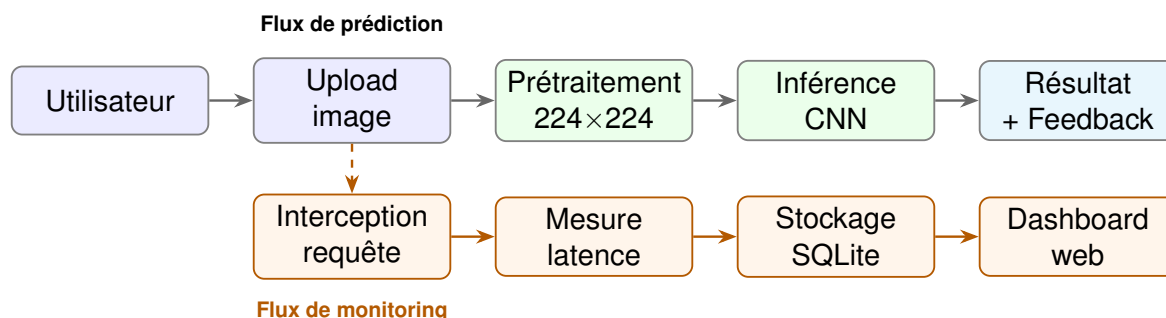


FIGURE 1 – Flux de traitement corrigé : prédiction (haut) et monitoring en parallèle (bas, orange pointillé).

3 Validation par tests automatisés

3.1 Stratégie de test

La suite de tests est organisée en deux classes `unittest : TestImageClassificationApp` couvre le flux de traitement des images (validation des extensions, prétraitement, encodage, routes Flask, feedback) ; `TestModelIntegration` vérifie la compatibilité entre le prétraitement et l'interface du modèle. Les tests utilisent des mocks pour simuler le modèle Keras sans dépendre du fichier `.h5`.

Le test critique (`test_preprocess_from_pil_shape_correction`) soumet une image synthétique 600×600 et vérifie que la sortie a bien la forme $(1, 224, 224, 3)$ attendue par le modèle. Son code est reproduit en Annexe B.

3.2 Résultats

Catégorie de test	Résultat
Validation des extensions de fichier	5/5 réussis
Redimensionnement et prétraitement	3/3 réussis
Fonctionnement des routes Flask	3/3 réussis
Feedback utilisateur	1/1 réussi
Intégration modèle (mock)	1/1 réussi
Total (durée : 18,4 s)	12/12 réussis

TABLE 3 – Résultats de la suite de 12 tests automatisés.

L'exécution sans erreur de l'ensemble de la suite confirme la correction du bug de redimensionnement, la robustesse du prétraitement pour toutes les tailles d'entrée testées (100×100 à 1024×768), et la stabilité des routes Flask.

4 Observabilité : journalisation, alerting et monitoring

4.1 Journalisation

La journalisation est assurée par la bibliothèque standard `logging` [7], configurée avec deux *handlers* : un fichier `app.log` et la sortie console. Trois niveaux sont utilisés selon la gravité : `logger.info()` pour l'état des services (ex. connexion MongoDB établie), `logger.warning()` pour les situations dégradées sans arrêt du service (ex. variable d'environnement absente), et `logger.error()` pour les exceptions avec traceback (ex. erreur de connexion à Atlas).

4.2 Alerting

En production, un système d'alerte email est activé via `SMTPHandler` (bibliothèque `logging.handlers`). Tout log de niveau `ERROR` ou supérieur déclenche automatiquement l'envoi d'un email avec sujet « ERREUR CRITIQUE -- App Classification » vers l'adresse définie dans la variable d'environnement `ALERT_EMAIL_RECIPIENT`. Les credentials SMTP sont chargés depuis les variables d'environnement, jamais codés en dur.

4.3 Monitoring

`Flask-MonitoringDashboard` est monté sur le endpoint `/dashboard`. Il intercepte automatiquement chaque requête HTTP et enregistre dans une base SQLite locale : le temps de réponse par endpoint, le code HTTP retourné, l'utilisation horaire et la comparaison multi-version. Des scripts Python dédiés (`monitoring/`) permettent d'extraire ces métriques pour une analyse hors ligne (détection de régressions, pics de latence).

5 Feedback loop et perspectives de ré-entraînement

5.1 Collecte des feedbacks

Après chaque prédiction, l'utilisateur peut corriger le label via un appel AJAX (`POST /feedback`). Le backend valide les champs reçus et insère dans MongoDB Atlas un document structuré contenant l'image encodée en base64, le label prédit avec sa confiance, le label corrigé par l'utilisateur, un booléen `is_correct` et la version du modèle ayant effectué la prédiction.

5.2 Stratégie de ré-entraînement sans fuite de données

Un script `cron` quotidien vérifie le nombre de feedbacks corrigés. Dès qu'un seuil (ex. 100 observations) est atteint, un pipeline de ré-entraînement est déclenché. La stratégie retenue **gèle le jeu de test initial** : seul l'ensemble d'entraînement est étendu avec les nouvelles données MongoDB, garantissant la comparabilité des métriques entre versions du modèle et éliminant tout risque de fuite de données (*data leakage*).

Références

- [1] Bogdan ANTONY et Patrick VISSER. *Flask Monitoring Dashboard – Automatically monitor your Flask web service*. Version 3.1.0. Consulté en mars 2026. 2024. URL : <https://flask-monitoringdashboard.readthedocs.io>.
- [2] François CHOLLET et al. *Keras – Deep Learning for humans*. Version 2.13.1. Consulté en mars 2026. 2024. URL : <https://keras.io>.
- [3] Alex CLARK et CONTRIBUTORS. *Pillow – Python Imaging Library (Fork)*. Version 10.x. Consulté en mars 2026. 2024. URL : <https://pillow.readthedocs.io>.
- [4] GUNICORN CONTRIBUTORS. *Gunicorn – Python WSGI HTTP Server for UNIX*. Consulté en mars 2026. 2024. URL : <https://gunicorn.org>.
- [5] MONGODB, INC. *MongoDB Atlas Documentation*. Consulté en mars 2026. 2024. URL : <https://www.mongodb.com/docs/atlas/>.
- [6] Pallets PROJECTS. *Flask Documentation – A lightweight WSGI web application framework*. Version 2.3.3. Consulté en mars 2026. 2024. URL : <https://flask.palletsprojects.com>.
- [7] PYTHON SOFTWARE FOUNDATION. *logging – Logging facility for Python*. Python 3.11. Consulté en mars 2026. 2024. URL : <https://docs.python.org/3/library/logging.html>.
- [8] PYTHON SOFTWARE FOUNDATION. *unittest – Unit testing framework*. Python 3.11. Consulté en mars 2026. 2024. URL : <https://docs.python.org/3/library/unittest.html>.

Annexes

A — Fonction preprocess_from_pil() : avant et après correctifs

Version originale (deux bugs) :

```
def preprocess_from_pil(pil_img: Image.Image) -> np.ndarray:
    img = pil_img.convert("RGB")
    # BUG 1 : pas de resize -> ValueError shape incompatible
    # BUG 2 : normalisation / 255.0 alors que le modele contient
    #         deja une couche Rescaling(1/255) -> double normalisation
    img_array = np.asarray(img, dtype=np.float32) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

Version finale corrigée (deux correctifs intégrés) :

```
# Chargement du modele et extraction dynamique des dimensions
model = keras.saving.load_model(MODEL_PATH, compile=False)
input_shape = model.input_shape          # ex. (None, 224, 224, 3)
MODEL_HEIGHT, MODEL_WIDTH = input_shape[1], input_shape[2]
# Note : model.layers[2] est une couche Rescaling(scale=1/255)
# => la normalisation est deja integree dans le modele

def preprocess_from_pil(pil_img: Image.Image) -> np.ndarray:
    """Prepare une image PIL pour l'inference Keras.

    Args:
        pil_img: Image PIL source, taille quelconque.

    Returns:
        np.ndarray de forme (1, MODEL_HEIGHT, MODEL_WIDTH, 3),
        dtype float32, valeurs dans [0, 255] -- la normalisation
        est delegue a la couche Rescaling du modele.
    """
    # 1. Conversion RGB
    img = pil_img.convert("RGB")
    # 2. CORRECTIF 1 : redimensionnement aux dimensions du modele
    img = img.resize((MODEL_WIDTH, MODEL_HEIGHT),
                     Image.Resampling.LANCZOS)
    # 3. Conversion en array float32
    # CORRECTIF 2 : suppression du / 255.0
    # (la couche Rescaling du modele s'en charge)
    img_array = np.asarray(img, dtype=np.float32)
    # 4. Ajout de l'axe batch
    img_array = np.expand_dims(img_array, axis=0)
    return img_array    # shape : (1, 224, 224, 3), valeurs in [0, 255]
```

B — Tests unitaires : vérification du redimensionnement

```
1 class TestImageClassificationApp(unittest.TestCase):
2
3     def test_preprocess_from_pil_shape_correction(self):
4         """Test critique : le preprocessing doit redimensionner
5         toute image a la forme attendue par le modele."""
6         test_img = Image.new('RGB', (600, 600), color='blue')
7         processed = preprocess_from_pil(test_img)
8
9         expected_shape = (1, 224, 224, 3)
10        self.assertEqual(
11            processed.shape, expected_shape,
12            f"Forme attendue {expected_shape}, obtenue {processed.shape}"
13        )
14        self.assertEqual(
15            processed.dtype, np.float32,
16            "Le type de donnees doit etre float32"
17        )
18
19    def test_preprocess_different_input_sizes(self):
20        """Le preprocessing doit fonctionner pour toute taille d'entree."""
21        test_sizes = [(100, 100), (300, 200), (800, 600), (1024, 768)]
22        for width, height in test_sizes:
23            with self.subTest(size=(width, height)):
24                img = Image.new('RGB', (width, height), color='green')
25                processed = preprocess_from_pil(img)
26                self.assertEqual(
27                    processed.shape, (1, 224, 224, 3),
28                    f"Taille {width}x{height} mal redimensionnee"
29                )
```

C — Test unitaire : détection de la double normalisation

```
1 class TestPreprocessNormalization(unittest.TestCase):
2     """Verifie que le preprocessing ne normalise pas les pixels,
3     la couche Rescaling du modele s'en chargeant."""
4
5     def test_no_manual_normalization(self):
6         """Les valeurs de sortie doivent rester dans [0, 255], pas [0, 1]."""
7         # Image synthetique avec des pixels blancs (valeur max = 255)
8         test_img = Image.new('RGB', (224, 224), color=(255, 255, 255))
9         processed = preprocess_from_pil(test_img)
10
11        # Si / 255.0 est present -> max ~= 1.0
12        # Si absent (correct) -> max ~= 255.0
13        self.assertGreater(
```

```

14         processed.max(), 1.0,
15         "Les pixels ne doivent PAS etre normalises dans le preprocessing "
16         "(la couche Rescaling du modele s'en charge).")
17     )
18     self.assertAlmostEqual(
19         float(processed.max()), 255.0, delta=1.0,
20         msg="Les pixels blancs doivent valoir ~255 apres preprocessing."
21     )
22
23     def test_rescaling_layer_present_in_model(self):
24         """Verifie que le modele contient bien une couche Rescaling(1/255)."""
25         rescaling_layers = [
26             l for l in model.layers
27             if l.__class__.__name__ == 'Rescaling'
28         ]
29         self.assertGreater(
30             len(rescaling_layers), 0,
31             "Le modele doit contenir au moins une couche Rescaling."
32         )
33         scale = rescaling_layers[0].get_config().get('scale', None)
34         self.assertIsNotNone(scale)
35         self.assertAlmostEqual(
36             scale, 1 / 255.0, places=6,
37             msg="La couche Rescaling doit avoir scale=1/255."
38         )
39
40     def test_no_double_normalization_effect(self):
41         """Verifie que deux images differentes produisent des predictions differentes
42         (symptome de la double normalisation : predictions identiques)."""
43         img_desert = Image.open("images_to_test/desert_96.jpg")
44         img_forest = Image.open("images_to_test/forest_42.jpg")
45         pred_desert = model.predict(preprocess_from_pil(img_desert), verbose=0)
46         pred_forest = model.predict(preprocess_from_pil(img_forest), verbose=0)
47         # Les vecteurs de probabilites ne doivent pas etre identiques
48         self.assertFalse(
49             np.allclose(pred_desert, pred_forest, atol=1e-3),
50             "Les predictions pour des images differentes doivent differer "
51             "(predictions identiques = symptome de double normalisation)."
52         )

```

```

1 class TestImageClassificationApp(unittest.TestCase):
2
3     def test_preprocess_from_pil_shape_correction(self):
4         """Test critique : le preprocessing doit redimensionner
5         toute image a la forme attendue par le modele."""
6         # Image synthetique 600x600 (taille qui declenchait le bug)
7         test_img = Image.new('RGB', (600, 600), color='blue')
8         processed = preprocess_from_pil(test_img)

```

```

9
10     expected_shape = (1, 224, 224, 3)
11     self.assertEqual(
12         processed.shape,
13         expected_shape,
14         f"Forme attendue {expected_shape}, obtenue {processed.shape}"
15     )
16     self.assertEqual(
17         processed.dtype,
18         np.float32,
19         "Le type de donnees doit etre float32"
20     )
21     # Verification de la normalisation [0, 1]
22     self.assertGreaterEqual(processed.min(), 0.0)
23     self.assertLessEqual(processed.max(), 1.0)
24
25     def test_preprocess_different_input_sizes(self):
26         """Le preprocessing doit fonctionner pour toute taille d'entree."""
27         test_sizes = [(100, 100), (300, 200), (800, 600), (1024, 768)]
28         for width, height in test_sizes:
29             with self.subTest(size=(width, height)):
30                 img = Image.new('RGB', (width, height), color='green')
31                 processed = preprocess_from_pil(img)
32                 self.assertEqual(
33                     processed.shape, (1, 224, 224, 3),
34                     f"Taille {width}x{height} mal redimensionnee"
35                 )

```
